Technical Interview Task: Secure Note-Taking Application

1. Application Overview

Build a simple REST API accompanied by a functional frontend. The visual style of the frontend is not a priority; the focus should be on functionality and integration.

Core Objective: Create a note-taking platform with secure authentication and role-based access control.

2. Roles & Permissions

Implement the following roles with specific access rights:

- User:
    - Can create, update, delete, and view a list of their own notes.
- Admin:
    - Inherits all User capabilities.
    - Can manage users (add, remove, update, and list all users).
    - Can view everyone's notes.

3. Technical Stack Requirements

You are required to use the following technologies and standards:

- Database: **MongoDB** with **Mongoose**.
- Authentication: **JWT** (JSON Web Tokens).
- Security: Implement secure **password hashing**.

4. Database Indexing & API Optimization

Note: You must use the `schema.index` method for defining indexes in your code so they are visible during review.

A. General Optimization

- Pagination: Implement pagination for all list operations in the REST API.
- List Operation Indexing: Create proper indexes to support all list views (e.g., a user listing their notes, an admin listing users).
- Read Operation Indexing: Ensure all `GET` operations (e.g., fetching a user profile or a specific note) are supported by indexes.

B. Aggregation Tasks

You must solve the following specific scenarios using MongoDB Aggregation Pipelines. Ensure all pipeline queries are supported by appropriate indexes.

- Scenario 1: Group by Interests
  - *Context:* User profiles contain a list of interests (e.g., `['chess', 'reading']`).
  - *Task:* specific view to see users grouped by interests.
  - *Constraint:* You must use exactly one `collection.aggregate()` call. Do not use any other methods.
- Scenario 2: User Posts ($lookup)
  - *Context:* Users can write posts which are stored in a separate `Posts` collection. These posts are visible to everyone.
  - *Task:* Retrieve all posts belonging to a particular user.
  - *Constraint:* Use a single aggregation pipeline with a `$lookup` stage.

5. Critical Constraint

DO NOT MAKE ANY UNNECESSARY INDEXES. You will be evaluated on the efficiency of your indexing strategy. Only create indexes that are strictly required to support the queries and aggregations described above.

DO NOT USE ANY TEMPLATES. DO NOT waste time improving the frontend**.** This is a backend task. **Learn Index & Aggregation** and **take your time** to submit the task.